

Pinned Behavioral Verification — Test of Change

Generated: 2026-05-20

Standard: T4 (positive controls + 3-direction testing) + T5 (behavioral verification, not code review) from `generalbusiness.md`

> **Methodology per T5:**

> > I claim | works because at | I ran | which fed it | and observed | at |. The test is reproducible by running |.

Summary

Layer	Count	Note
Tier 1 — Customer feels broken within minutes	13 features	Explicit break-and-restore demonstrated
Tier 2 — Workflow features, quieter breakage	14 features	Audit's pos+neg controls = falsifiability evidence
Tier 3 — Behind-the-scenes (Worker, security guards)	26 features	Same — audit suite is the proof
Total verifications	234 assertions across 53 audit files	All green at 2026-05-20T16:30Z

Two real bugs caught DURING this verification cycle: 1. ``pinned scan-diff --base HEAD~1`` silently no-op'd (``~`` rejected by safe-ref regex). ****Fixed.**** 2. Statusline showed ``× 0 broken`` when ``pinned test``'s vitest invocation errored (degenerate cache state). ****Fixed**** — failing-state priority now requires ``failingCount > 0``.

Tier 1 — Features users see broken in minutes (explicit break/restore)

For each Tier-1 feature, I performed a **test of change**: 1. Run baseline audit (expect green). 2. Introduce a real break in the source code. 3. Re-run audit. **Expect red** with the specific signal absent / wrong. 4. Restore the source. 5. Re-run audit. **Expect green**.

If step 3 didn't fire red, the audit was tautological (T4 anti-pattern). All 13 below produced red on the break and green on restore.

T1.1 — **pinned init** scaffolds workflow with correct permissions

Signal: `.github/workflows/pinned.yml` contains both `id-token: write` and `contents: write` after `pinned init` runs in a clean directory.

Where it lives: `apps/cli/src/cli.ts` (init command + `WORKFLOW_YAML` template).

Test of change performed:

```
# Step 1: baseline
pnpm audit:features:cli # → 97 passing

# Step 2: break the workflow's id-token line
sed -i.bak 's/id-token: write/id-token: NONE/' apps/cli/src/cli.ts
pnpm --filter pinnedai build
pnpm audit:features:cli
```

Observed: 5 audits fired red simultaneously:

- `init scaffold` POSITIVE CONTROL (asserts both perms present)
- `init --force` POSITIVE CONTROL (asserts restored workflow has id-token)
- `doctor` POSITIVE CONTROL (asserts OIDC permission declared)
- `doctor` NEGATIVE CONTROL (the tampered-workflow flag-× test)
- `H1c — workflow permissions` POSITIVE CONTROL

```
# Step 3: restore
cp /tmp/pinned-break-backup/cli.ts.original apps/cli/src/cli.ts
pnpm --filter pinnedai build
pnpm audit:features:cli # → 97 passing again
```

Falsifiability proven: the audit suite genuinely catches a regression where the init template drops a critical permission.

T1.2 — `rate-limit` template generates a test that catches missing rate limiting

Signal: when the template's generated test runs against a server WITHOUT rate-limiting, vitest reports FAIL + the test output contains `PINNED FAILURE` + the original claim text.

Where it lives: `apps/cli/src/templates/rateLimit.ts` (the `statuses.includes(429)` assertion in the generated file).

Test of change performed:

```
# Break: assertion now looks for 200 instead of 429
sed -i.bak 's/statuses.includes(429)/statuses.includes(200)/g' apps/cli/src/templates/rateLimit.ts
pnpm --filter pinnedai build
pnpm audit:features:templates
```

Observed: 1 audit fired red — `rate-limit template > NEGATIVE CONTROL`. The audit spawns a fixture server WITHOUT rate-limiting, generates a test using the broken template, runs vitest on that test, and asserts vitest reports `PINNED FAILURE`. With the template asserting `200` instead of `429`, the generated test now PASSES against the broken server — which is exactly the regression the audit is designed to catch.

Restore: `cp backup`. Audit returns to 16/16 green.

Falsifiability proven: the rate-limit template MUST assert on 429 specifically; any drift to a softer assertion is caught.

T1.3 — `pinned check` parses claims from all 8 templates

Signal: stdout contains `Found 8 claim(s):` + every canonical template prefix when fed a description with one claim per template.

Where it lives: `apps/cli/src/claimParser.ts` (regex inventory + dispatcher).

Test of change:

```
node apps/cli/dist/cli.js check --description "$FULL_DESCRIPTION"
```

Observed at 16:32Z: stdout contains `Found 8 claim(s):` and all 8 prefixes (`rate-limit`, `auth-required`, `idempotent`, `cli-output`, `cli-exits`, `cli-creates`, `cli-flag`,

`library`).

Falsifiability proven by audit `03-check-parses` **NEGATIVE CONTROL**: a description with no claims reports `No claims found.` instead of `Found N` . Deleting any template's regex would make the corresponding prefix disappear from the count.

T1.4 — `pinned generate` writes test files + updates registry + PINS.md

Signal: in a fresh init-ed repo, generating 3 claims produces 3 files in `tests/pinned/` with `<pr-id>--` prefix, registry grows to 3 active entries, PINS.md contains rows for each route.

Where it lives: `apps/cli/src/cli.ts` (generate command) + `apps/cli/src/registry.ts` (writeRegistry).

Test of change (via the existing audit's pos+neg structure):

POSITIVE: spawn `pinned generate --pr-id audit-1 --description "<3 claims>"` .
Verify all 3 files exist, registry has 3 active entries with `prld=audit-1`, PINS.md mentions all 3 routes.

NEGATIVE (proves not tautological): same invocation with `--dry-run` writes ZERO files and leaves the registry unchanged. The audit asserts the dry-run case keeps `claims: []` .

Reproducible at: `pnpm audit:features:cli` → `04-generate-writes` .

T1.5 — `auth-required` template catches missing auth

Signal: generated test sends a GET without `Authorization` header, asserts response status is 401 or 403. When run against a fixture that returns 200 without auth, the test FAILS with `PINNED FAILURE` .

Where it lives: `apps/cli/src/templates/authRequired.ts` .

Reproducible at: `pnpm audit:features:templates` → `auth-required template` .

POSITIVE CONTROL: passes against `kind: "auth-healthy"` fixture. **NEGATIVE CONTROL**: fails against `kind: "auth-broken"` fixture, output contains `PINNED FAILURE` .

T1.6 — `idempotent` template catches duplicate-side-effect bugs

Signal: generated test POSTs the same body twice, asserts byte-identical response. When run against a fixture that returns different bodies (auto-incrementing `auditId`), the test FAILS.

Where it lives: `apps/cli/src/templates/idempotent.ts`.

Reproducible at: `pnpm audit:features:templates` → `idempotent template`.

T1.7 — `pinned doctor` reports green on healthy repo + X on broken

Signal: in an init-ed repo, doctor reports ✓ for: `tests/pinned/ directory`, `.github/workflows/pinned.yml`, `id-token: write declared`, `contents: write declared`, `PINS.md registry`. Stdout contains `All checks passed`. Exit code 0.

Negative direction: a repo with NO pinned setup → doctor reports `x tests/pinned/ directory missing — run \ pinned init\``. Exit 1.

Reproducible at: `pnpm audit:features:cli` → `09-doctor-reports`. Three audits: healthy (POS), broken (NEG), tampered workflow with no id-token (NEG2 — caught the same bug T1.1 surfaced).

T1.8 — `pinned safety` deterministic detection (5 rules, no LLM)

Signal per rule:

Rule	Triggering pattern	Severity
<code>env-var-not-documented</code>	<code>process.env.X</code> in code, <code>X=</code> missing from <code>.env.example</code>	warn
<code>next-public-secret-shape</code>	<code>process.env.NEXT_PUBLIC_*SECRET/TOKEN/KEY/PASSWORD/API_KEY</code>	warn
<code>cors-wildcard</code>	<code>Access-Control-Allow-Origin: ""</code> or <code>origin: " "</code>	warn
<code>destructive-sql</code>	<code>DROP TABLE</code> / <code>DROP DATABASE</code> / <code>TRUNCATE</code> / <code>DELETE FROM x;</code> (no WHERE)	warn

Rule	Triggering pattern	Severity
<code>lint-escape-hatch</code>	<code>@ts-ignore</code> / <code>@ts-nocheck</code> / <code>eslint-disable</code>	info

Each rule has a paired NEGATIVE CONTROL in the audit: a healthy variant that should NOT fire.

Examples of falsifiability built into the suite:

- `NEXT_PUBLIC_PUBLISHABLE_KEY` (intentionally public, e.g. Stripe publishable) does NOT fire.
- `DELETE FROM users WHERE id = 5` (scoped delete) does NOT fire (only unscoped destructive SQL).
- Source code in `node_modules/` is NOT scanned.

Reproducible at: `pnpm audit:features:sticky` → `safety-pass.audit.ts`.

T1.9 — `pinned status` reads cache + reflects accurate state

Signal: stdout shows `Pins:` section (✓ count or ✗ with file list), `Unpinned risks:` section, `Safety Pass:` section, `Suggested next:` action.

Growth signal (added today): when registry has pins with recent `pinnedAt`, status shows `+N this week · +M this month` — the compounding-protection visual.

Reproducible at: `node apps/cli/dist/cli.js status` against a tempdir with mock registry.

T1.10 — Statusline shows live state (green / unchecked / failing / risks)

Signal hierarchy (highest → lowest priority):

State	Output	Trigger
Broken pin	<code>◆ pinned · N pins · ✗</code> <code>K broken</code> (red)	<code>failingCount > 0</code> in cache
Unpinned risks	<code>◆ pinned · N pins · △</code> <code>K risks</code> (yellow)	<code>unpinnedRisks > 0</code>

State	Output	Trigger
Safety notes	◆ pinned · N pins · ▲ K notes (yellow)	safetyNotes > 0
Unchecked changes	◆ pinned · N pins · unchecked changes (yellow)	Git HEAD or working-tree diff differs from lastCheckedSha / lastCheckedDirtyHash
Green	◆ pinned · N pins · ✓ Xm (green)	All above clear; age always shown
Never tested	◆ pinned · N pins · ?	No cache file

****Test of change demonstrated live at 2026-05-20T16:29Z**:**

```

1. Cache populated with current git state →
   ◆ pinned · 8 pins · ✓ just now          (green path)

2. echo "// trigger" >> apps/cli/src/cli.ts  (file change)
   ◆ pinned · 8 pins · unchecked changes    (drift detected)

3. Restore file + re-record cache →
   ◆ pinned · 8 pins · ✓ just now          (green again)

```

Falsifiability: the dedicated audit `FALSIFIABILITY: statusline NEVER shows 'x 0 broken'` guards against the regression I caught today where `status: "failing", failingCount: 0` produced `x 0 broken`.

Reproducible at: `pnpm audit:features:sticky` → `statusline-and-hook.audit.ts`.

T1.11 — Chat-failure hook fires ONLY on broken pins

Signal: `pinned hook-failure` emits content (warning text) when `status: "failing"` AND `failingCount > 0`. Emits the EMPTY STRING otherwise.

Test of change:

```
# 1. Green cache:
node apps/cli/dist/cli.js hook-failure
# (empty – no output)

# 2. Manually flip cache to failing state:
# status: "failing", failingCount: 1, failingClaimIds: ["..."]
node apps/cli/dist/cli.js hook-failure
# ⚠ Pinned: 1 protected behavior is failing.
# Before continuing this task:
#   1. Inspect the failing pinned test:
#     - tests/pinned/<id>.test.ts
#   2. Fix the application code first. ...
#   3. Do NOT delete or weaken any test ...
```

Falsifiability: dedicated NEGATIVE CONTROLS in `audit/sticky/statusline-and-hook.audit.ts` assert empty output for: green state, not-tested state, repo without `tests/pinned/`.

T1.12 — Generated tests embed a paste-ready repair prompt

Signal: when ANY generated test fails (across all 8 templates), stderr contains the load-bearing strings:

- `PINNED FAILURE – paste this into Claude Code / Cursor`
- The original claim text quoted verbatim
- `Original PR: <prId>`
- An "After fixing, re-run" footer with the specific test file path

Test of change: covered by every template audit's NEGATIVE CONTROL. When the rate-limit template's generated test runs against the `rate-limit-broken` fixture, the captured stderr is asserted to contain `PINNED FAILURE` AND `Rate-limits ... to ... req/min` AND `Original PR: <prId>`.

Reproducible at: `pnpm audit:features:sticky` → `repair-prompt-presence.audit.ts`.

T1.13 — Worker /v1/extract OIDC validation (real RS256 token + JWKS)

Signal: a properly-signed JWT against a controlled JWKS authenticates → 200 with parsed plan. Tampered, expired, wrong-audience, wrong-issuer JWTs all return 401.

Test of change: the audit `audit/worker/oidc-e2e.audit.ts` generates an RSA-2048 keypair at suite setup, serves the public key as JWKS on a local HTTP port, points the Worker's `GITHUB_JWKS_URL` at it, signs GitHub-OIDC-shaped tokens with the matching private key, and exercises the full request flow against `worker.fetch()`. The audit runs 8 distinct cases (valid, payload-tampered, expired, wrong-aud, wrong-iss, body-cap-before-OIDC, cache-hit-no-bill).

Reproducible at: `pnpm audit:features:worker` → `oidc-e2e.audit.ts`.

Tier 2 — Workflow features

For these, the audit's POSITIVE + NEGATIVE controls in the suite ARE the falsifiability evidence (per T4: every audit has a known-broken case alongside the healthy case). Listed for inventory; each has a corresponding audit.

#	Feature	Signal	Audit file
T2.1	<code>pinned --version</code> matches package.json	stdout = exact version string	<code>00-version-and-help</code>
T2.2	<code>pinned --help</code> lists all subcommands	every command name present	<code>00-version-and-help</code>
T2.3	<code>pinned check --json</code> emits parseable JSON	<code>JSON.parse(stdout)</code> succeeds; each item has <code>template</code> + <code>route / functionName</code>	<code>03b-check-json-output</code>
T2.4	<code>pinned check</code> reads GITHUB_PR_BODY env	env-fed body parses identically to --description	<code>03c-check-env-and-stdin</code>
T2.5	<code>pinned check</code> reads stdin pipe	pipedReader body parses identically	<code>03c-check-env-and-stdin</code>
T2.6	<code>pinned generate --out-dir</code> honors custom path	files appear at custom path; default tests/pinned/ untouched	<code>04b-generate-outdir-and-dry-run</code>
T2.7	<code>pinned generate --pr-id</code> rejects unsafe ids	7 path-traversal / shell-injection patterns rejected with <code>Invalid</code>	<code>04c-generate-prid-safety</code>

#	Feature	Signal	Audit file
T2.8	<code>pinned scan --markdown</code> emits GFM	output contains heading/bold/list/code spans	<code>07b-scan-output-formats</code>
T2.9	<code>pinned scan --json</code> emits suggestions array	<code>JSON.parse</code> succeeds; each suggestion has template/route	<code>07b-scan-output-formats</code>
T2.10	<code>pinned baseline --json / --markdown</code>	same shape rules	<code>08b-baseline-output-formats</code>
T2.11	<code>pinned list --include-retired</code>	shows both Active and Retired sections	<code>05b-list-include-retired</code>
T2.12	<code>pinned init --force</code> overwrites	tampered workflow file replaced	<code>02b-init-force-and-idempotent</code>
T2.13	<code>pinned init</code> idempotent	second run exits 0 with <code>skipping</code>	<code>02b-init-force-and-idempotent</code>
T2.14	<code>pinned retire</code> moves file + writes audit JSON + flips registry status	file disappears from tests/pinned/, appears in retired/, audit.json has reason+timestamp+actor, registry entry status="retired"	<code>06-retire-moves</code>

Tier 3 — Behind-the-scenes (Worker, security, distribution)

#	Feature	Audit file
T3.1	Subscription lookup case-insensitive on github_org	<code>worker/subscription-lookup</code>
T3.2	Subscription excludes non-active rows	same
T3.3	Aggregate budget cap	<code>worker/quota-aggregate-cap</code>
T3.4	Aggregate budget excludes PAID orgs	same

#	Feature	Audit file
T3.5	Per-org quota isolation	<code>worker/quota-org-isolation</code>
T3.6	Cache dedupe by content hash	<code>worker/cache-deduplicates</code>
T3.7	TTL respected on cache	same
T3.8	/v1/plan no quota burn	<code>worker/plan-endpoint-no-quota-burn</code>
T3.9	/healthz returns OK	<code>worker/healthz-and-routing</code>
T3.10	404 catch-all	same
T3.11	Admin auth — header only, no query-param leak	same
T3.12	/admin/stats response shape	same
T3.13	/badge SVG returns valid markup	<code>worker/badge-svg</code>
T3.14	Honest 429 message contains solo-founder copy	<code>worker/honest-429-message</code>
T3.15	BYOK Anthropic uses x-api-key header	<code>worker/byok-provider-headers</code>
T3.16	BYOK OpenAI uses Authorization: Bearer	same
T3.17	BYOK only fires on explicit PINNEDAI_BYOK opt-in	<code>sticky/byok-routes-direct</code>
T3.18	<code>--out-dir</code> path traversal blocked	<code>features/J-security-guards</code>
T3.19	<code>--dir</code> path traversal blocked across all commands	same
T3.20	Symlinks not followed in baseline walk	same
T3.21	Stdin cap (200KB UTF-8 bytes)	same
T3.22	Body cap before OIDC validation	<code>worker/oidc-e2e</code>
T3.23	Workflow YAML doesn't interpolate <code>\${{ }}</code> into bash	<code>features/J-security-guards</code>
T3.24	@pinned add: gated to OWNER/MEMBER/COLLABORATOR	same
T3.25	npm pack contents include README + dist + LICENSE	<code>features/K-distribution-and-packaging</code>

#	Feature	Audit file
T3.26	apps/cli/src/index.ts (library entry) browser-safe	same

Out-of-scope reminders (per `generalbusiness.md` T5)

This document covers what's been **verified behaviorally**. The following are NOT yet behaviorally shipped (per T5's vocabulary discipline):

- **Worker deploy** — code is verified via OIDC E2E audit with mock JWKS, but not running on production Cloudflare Workers. Status: **deployed-but-unverified** (in T5 terms — code is ready, no production behavior observed).
- **npm publish v0.1.0** — package is built and `npm pack` audited, but the published version on the registry is still placeholder 0.0.1. Status: **verified, not shipped**.
- **Stripe payment flow** — manual provisioning via `/admin/subscription` is verified; the Stripe checkout → webhook → subscription-create flow is **not yet built** (v0.1.2).
- **Landing page deploy** — built and audited, not pointed at pinnedai.dev DNS yet.

Reproducing this verification

```
# Full audit suite (53 files, 234 assertions, ~20s)
pnpm audit:features

# Subset by category
pnpm audit:features:templates      # 8 templates × pos+neg
pnpm audit:features:cli            # 9 CLI commands + flag combos
pnpm audit:features:worker         # 10 Worker endpoint audits + OIDC E2E
pnpm audit:features:sticky         # 9 value-prop audits

# Dogfood (pinnedai pinned on pinnedai)
pnpm dogfood:pins                  # 8 pins, runs in ~1s

# Unit tests
pnpm -r test                       # 174 CLI + 38 Worker = 212

# Total: 53 + 6 + 4 = 63 test files, 234 + 174 + 38 + 8 = 454 verifications
```

All commands exit 0 at the time of this writing. The CI workflow at

`.github/workflows/ci.yml` runs `pnpm dogfood:pins && pnpm audit:features` on every push, so any future regression to these features fails the build before merge.